Adattamento del Framework B-Human e Sviluppo di Comportamenti Cooperativi sul Mezzo Campo del Laboratorio DIMES

Relazione tecnica delle attività svolte nell'ambito del Percorso di Eccellenza

Francesco Romeo[1,‡]

[1]DIMES, Università della Calabria, Rende, Italia.
Matricola: ‡ *264409*.

Sommario

La presente relazione documenta le attività svolte nell'ambito del Percorso di Eccellenza, all'interno del quale 50 ore sono state impiegate nel progetto RoboCup del DIMES. Il lavoro ha riguardato l'aggiornamento del software a bordo dei robot NAO, la loro configurazione all'interno della rete locale del laboratorio e il deploy del codice B-Human, basato su una versione custom sviluppata dalla nostra università. Successivamente, sono state adattate alle caratteristiche del campo UnicalMini alcune delle skill comportamentali precedentemente progettate dai colleghi che hanno lavorato al progetto, ottimizzandone i parametri e il funzionamento sul nuovo ambiente di test.

1 Introduzione

Le attività documentate in questa relazione descrivono il lavoro svolto durante le 50 ore dedicate al progetto RoboCup del Dipartimento di Ingegneria Informatica, Modellistica, Elettronica e Sistemistica (DIMES), nell'ambito del Percorso di Eccellenza. L'obiettivo principale è stato il ripristino operativo, la calibrazione e l'avanzamento del software di controllo per robot umanoidi NAO V6, basato su una versione del framework B-Human (Code Release 2024) [1] personalizzata dalla nostra università.

Il lavoro si è articolato in diverse fasi consequenziali. La prima parte ha riguardato la messa a punto dell'hardware e dell'ambiente operativo: l'aggiornamento del

sistema operativo dei robot, la loro configurazione all'interno della sottorete locale del laboratorio e la verifica del deploy del codice sorgente compilato a bordo macchina.

Dato che queste fasi preliminari di configurazione sistemistica sono già state ampiamente documentate e standardizzate a beneficio del laboratorio, si rimanda per ogni dettaglio operativo alle rispettive guide su Notion:

- *Setup e Configurazione Iniziale NAO v6*: Guida Setup e Update NAO v6.
- *Deploy B-Human to NAO*: Guida Deploy B-Human.

Per questa ragione, la presente relazione si concentra sugli aspetti successivi:

1. ***Modellazione e configurazione del mezzo campo del laboratorio***: definizione delle dimensioni geometriche, delle linee e dei riferimenti del campo ridotto del laboratorio, sia a livello di simulatore (SimRobot) sia per i moduli di localizzazione e percezione dei robot reali.
2. ***Adattamento e calibrazione delle skill comportamentali***: analisi, porting e ottimizzazione sul campo UnicalMini delle skill sviluppate dai colleghi che hanno lavorato in precedenza al progetto, risolvendo i problemi di stallo, calibrando i movimenti negli spazi ristretti e garantendo la corretta interazione tra i ruoli.

2 Modellazione e Configurazione del Mezzo Campo

2.1 Motivazione e Definizione del Problema

Il framework B-Human è nativamente configurato per operare su campi ufficiali RoboCup SPL di dimensioni standard (9×6 m), caratterizzati da due metà campo simmetriche e due porte regolamentari. All'interno del laboratorio di robotica del DIMES, per ragioni di spazio disponibile, è presente invece una superficie ridotta corrispondente a un singolo mezzo campo, denominato `UnicalMini`, di circa 3.22 m di lunghezza per 4.40 m di larghezza, dotato di una sola porta e delimitato a ridosso da pareti. L'utilizzo del software standard su un ambiente così compatto e asimmetrico comporta evidenti difficoltà operative: i robot cercano riferimenti geometrici inesistenti mandando in crisi la localizzazione, la percezione della palla viene falsata dai confini ristretti e le traiettorie di movimento calcolate portano i robot a scontrarsi con le pareti del laboratorio. Per risolvere queste problematiche alla radice, è stato necessario introdurre una nuova configurazione (*location*) dedicata, denominata `UnicalMini`, adattando i parametri geometrici e sensoriali sia a bordo macchina sia nel simulatore SimRobot.

2.2 Rilievo delle Misure del Campo

Per parametrizzare correttamente il mezzo campo, è stata effettuata una fase di misurazione diretta degli spazi all'interno del laboratorio. I riferimenti geometrici rilevati corrispondono allo schema standard adottato dal framework B-Human per la descrizione dei campi di gioco, illustrato in Figura 1. In particolare, il rilievo ha interessato tutte le quote longitudinali lungo l'asse x (linea di metà campo, limiti dell'area di rigore e di porta, dischetto, linea di fondo e pali) e trasversali lungo l'asse y (linee laterali, larghezza delle aree e luce della porta). Le misurazioni sul campo reale sono

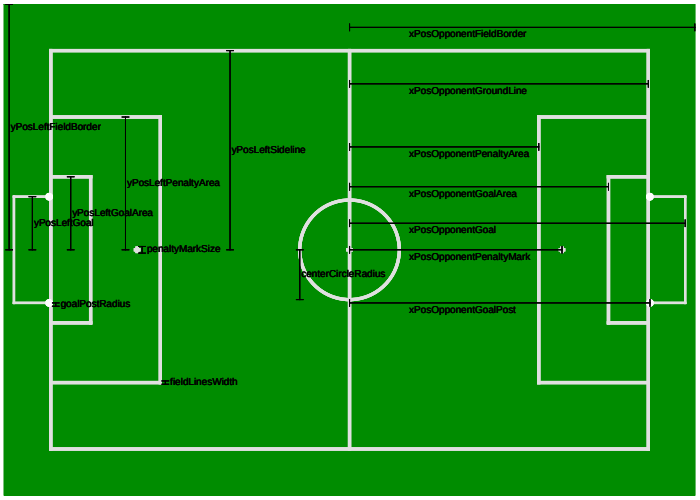


Figura 1: Schema dei parametri geometrici longitudinali e trasversali utilizzati per la calibrazione del campo in B-Human.

state acquisite tramite l'applicazione *Metro* di iOS. Trattandosi di uno strumento di stima basato su realtà aumentata, i valori ottenuti presentano un margine di approssimazione fisiologico e non una precisione millimetrica. Ciononostante, le tolleranze risultano pienamente adeguate per la navigazione e il gioco dei robot.

Tali valori sono stati codificati nel file principale `fieldDimensions.cfg`. A completamento della cartella di configurazione `Config/Locations/UnicalMini/`, sono stati adattati in modo sintetico anche gli altri moduli chiave del sistema:

- `sideInformationProvider.cfg`**:** forza il robot a considerare unicamente la metà campo utile, eliminando alla radice il rischio che il sistema ribalti di 180° il proprio orientamento stimato cercando una porta assente;
- `selfLocator.cfg`**:** ricalibra le tolleranze di odometria e i parametri del filtro particellare per velocizzare la convergenza della posizione negli spazi ristretti;
- `ballPerceptFilter.cfg`**:** introduce un limite di distanza massima sulla percezione della palla per scartare riflessi del pavimento ed evitare falsi positivi al di fuori del campo;
- `optionDemoPass*.cfg`**:** riduce la potenza dei calci e adegua i raggi di ricezione, evitando che la palla impatti ad alta velocità contro le pareti del laboratorio.

2.3 Modellazione e Replicazione in SimRobot

Prima di eseguire il codice a bordo dei robot fisici, il mezzo campo è stato replicato all'interno del simulatore **SimRobot**, l'ambiente ufficiale di simulazione fisica di B-Human basato sul motore dinamico ODE (*Open Dynamics Engine*). Questo passaggio è essenziale per validare le logiche decisionali e la cinematica in totale sicurezza. L'infrastruttura virtuale di `UnicalMini` comprende:

- **Modello 3D del campo (`UnicalMiniField.rsi2`):** definisce la geometria tridimensionale della scena, riproducendo il manto verde, le linee di gioco con le dimensioni ricavate dal rilievo e i corpi rigidi per i pali e la traversa dell'unica porta;
- **File di scena (`.ros2`) e script (`.con`):** configurano le condizioni iniziali della simulazione e automatizzano l'apertura delle viste di debug e dei grafici di telemetria.

L'infrastruttura virtuale e l'interfaccia di simulazione risultante sono illustrate in Figura 2.



Figura 2: Simulazione del mezzo campo *UnicalMini* in SimRobot: rendering tridimensionale con modello fisico del robot e della porta (a sinistra) e vista di stato 2D (`worldState`) con tracciamento della palla e cono di visione (a destra).

Nel laboratorio reale il terreno di gioco è delimitato da muri e dietro la porta non è presente una rete morbida, bensì direttamente la parete della stanza. Per replicare fedelmente tale dinamica anche nell'ambiente simulato, sia le quattro pareti perimetrali sia la barriera di fondo della porta sono state modellate come corpi solidi con materiale `wall` all'interno del file `UnicalMiniField.rsi2`. Nel motore fisico del simulatore (`Util/SimRobot/Src/SimRobotCore2/Simulation/Simulation.cpp`) è stato quindi abilitato il rimbalzo elastico (`dContactBounce`) per la palla contro tali superfici, impostando un coefficiente di restituzione `bounce = 0.65f` e una soglia minima di velocità `bounce_vel = 0.05f` (5 cm/s), entrambi direttamente modificabili per eventuali ricalibrazioni future.

L'adattamento della scena ha tuttavia messo in luce due criticità specifiche legate all'architettura di simulazione di B-Human:

1. La prima ha riguardato un *falso positivo nella percezione*: durante i test, i robot scambiavano sistematicamente la croce bianca del calcio di rigore per il pallone, tentando ripetutamente di calciarla. L'inconveniente è riconducibile al modello di computer vision (`BallAndPenaltyMarkPerceptor`), addestrato sulle marcature regolamentari dei campi ufficiali RoboCup: le proporzioni o dimensioni leggermente differenti della croce disegnata sul campo `UnicalMini` traevano in inganno la rete, che faticava a distinguerla dalla sfera. Poiché l'obiettivo della simulazione era validare le logiche decisionali e non riaddestrare il modello visivo, negli script di avvio (`.con`) si è scelto di disaccoppiare la visione della palla affidandosi direttamente all'oracolo del motore fisico (`OracledWorldModelProvider`). In questo modo il robot mantiene attivo il proprio filtro di localizzazione sul campo (`SelfLocator`), ma riceve le coordinate esatte del pallone senza essere tratto in inganno dalla marcatura a terra.
2. La seconda criticità ha interessato *il sistema di riferimento cartesiano interno dei robot* (`worldState`). Nelle regole standard RoboCup, la prima squadra difende la metà campo a coordinate negative ($x < 0$), mentre la seconda squadra difende quella positiva ($x > 0$). Per convenzione interna di B-Human, nominare i robot nella scena come `robot1` o `robot2` li associa automaticamente alla prima squadra; trovandoli collocati sul nostro mezzo campo a coordinate interamente positive ($x \in [0, 3220]$ mm), SimRobot applicava in automatico un ribaltamento speculare. I robot credevano quindi di trovarsi fuori dal campo e girati di spalle, interpretando i tiri verso la porta come allontanamenti della sfera. Per ovviare all'anomalia senza toccare il sorgente del simulatore, nel file `.ros2` i robot sono stati configurati con gli identificativi convenzionali della seconda squadra (ID superiori a 20): il Giocatore 1 alla porta è stato istanziato come `robot21` (maglia viola) e il Giocatore 2 a centrocampo come `robot22` (maglia nera). Essendo riconosciuti come membri della squadra del semiasse positivo, SimRobot preserva l'orientamento corretto del campo, assicurando la piena concordanza tra coordinate simulate e misurazioni sul campo reale.

File di scena e script di configurazione

A completamento dell'ambiente virtuale, per ciascun esercizio sono stati realizzati specifici file di scena (`.ros2`) e script di comando (`.con`) dedicati. I file `.ros2`, strutturati in formato XML, assemblano i modelli 3D del campo, del pallone e dei robot nel motore della simulazione e ne definiscono le condizioni iniziali di spawn; ad esempio, nello scenario del duello il portiere viene generato davanti alla porta rivolto a 180° verso il campo, mentre l'attaccante viene posizionato a centrocampo orientato a 0°. I file di script `.con` automatizzano invece l'inizializzazione della simulazione all'avvio: disattivano le regole e le interruzioni arbitrali della RoboCup (`ar <regola> off`) per consentire lo svolgimento continuo del drill e aprono le finestre di telemetria e le viste 2D del campo.

3 Adattamento e Calibrazione delle Skill Comportamentali

La logica decisionale e il controllo ad alto livello dei robot NAO in B-Human sono formalizzati mediante il linguaggio **CABSL** (*C-based Agent Behavioral Specification Language*), un framework a macchine a stati finiti gerarchiche. Ogni comportamento (*skill*) definisce un grafo di stati discreti: a ogni ciclo di esecuzione della cognizione, il sistema valuta le condizioni di transizione ed esegue le azioni associate, traducendole in richieste per il controllo motorio, per la postura e per il movimento del collo e della telecamera.

Prima di passare a comportamenti complessi con il pallone, il percorso sperimentale è partito dal porting e dalla validazione delle skill precedentemente realizzate dai colleghi, procedendo in modo incrementale dalla cinematica di base fino alla progettazione di comportamenti cooperativi avanzati a ciclo continuo.

3.1 Le Skill Singole di Base: Validazione Cinematica e Tiro in Porta

La prima verifica sul campo reale ha riguardato la skill `DemoWalkSquare` sviluppata dai colleghi. Il comportamento, privo di interazione con il pallone, guida il NAO lungo un percorso quadrato alternando tratti rettilinei a rotazioni di 90°. Questa prova preliminare è servita essenzialmente a verificare che l'odometria interna e la percezione del robot rispondessero in maniera coerente con le geometrie ridotte del laboratorio, accertando la stabilità della camminata prima di introdurre il gioco con la palla.

Successivamente è stata portata sul mezzo campo la skill di tiro `DemoScoreGoal`, nella quale il robot cerca la palla, vi si avvicina, mira verso la porta avversaria tramite `KickAtGoal` e calcia. L'adattamento alle geometrie di `UnicalMini` ha richiesto l'aggiornamento dei riferimenti della linea di porta (`xPosOpponentGoalLine` a 3220 mm), affinché l'algoritmo calcolasse la direzione del tiro verso la luce effettiva della nostra porta. Inoltre, per evitare che il robot perdesse troppo tempo a cercare l'angolazione perfetta compiendo continui micro-passi di allineamento, è stato previsto un tempo massimo di attesa oltre il quale il robot smette di aggiustare la posizione ed esegue direttamente un calcio dritto in avanti.

La criticità principale della skill originale risiedeva tuttavia nella gestione del fine azione: una volta effettuato il tiro, il robot rimaneva attivo continuando a camminare in avanti verso il pallone, rischiando di impattare contro la porta o contro la parete posteriore della stanza. Per ovviare al problema è stata creata la variante `DemoScoreGoalAndStop`: dopo aver calciato, il robot attende sul posto (`Stand()`) osservando la sfera; se la palla attraversa la linea di porta tra i pali, il robot annuncia vocalmente il gol e transisce definitivamente nello stato terminale `done`, arrestando completamente la camminata in posizione eretta e con lo sguardo rivolto in avanti.

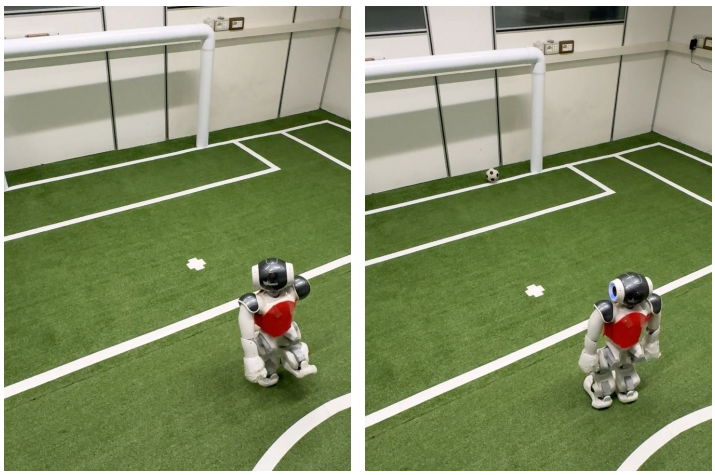


Figura 3: Esecuzione sul campo reale di `DemoScoreGoalAndStop`: allineamento e conduzione al tiro (a sinistra) e arresto di sicurezza nello stato `done` a gol avvenuto (a destra).

3.2 Il Duello Attaccante vs Portiere: Adattamento del Ciclo Continuo

Dopo aver validato i movimenti di base, l'attenzione si è spostata sullo scenario cooperativo a due robot che contrappone un attaccante e un portiere. Questo comportamento era già stato strutturato dai colleghi che hanno lavorato precedentemente al progetto con l'obiettivo di realizzare un ciclo continuo: l'attaccante calcia verso la porta, il portiere interviene e successivamente rinvia la palla a centrocampo, permettendo all'attaccante di riprendere il possesso e ripetere l'azione indefinitamente.

Inizialmente, per semplicità operativa e per validare separatamente i ruoli sul nuovo mezzo campo `UnicalMini`, è stata testata una versione a singola interazione (`DemoStrikerVSKeeperMini`) in cui l'azione si interrompeva al termine del tiro o della parata. Verificato il corretto intervento del portiere (tra tuffi laterali e parate a terra in ginocchio con `genuflectStand`), si è passati all'adattamento della logica a ciclo continuo completo (`DemoStrikerVSKeeperLoopMini`).

Il porting sul mezzo campo reale ha richiesto la risoluzione di diverse problematiche che impedivano la corretta chiusura del loop:

1. *Adattamento geometrico delle coordinate:* la logica originale si appoggiava su riferimenti simmetrici, mentre su `UnicalMini` l'intero spazio di gioco è mappato sul semiasse positivo ($x \in [0, 3220]$ mm). Sono state quindi ricalibrate tutte le soglie dimensionali: la posizione di guardia del portiere a $x = 3000$ mm, il limite dell'area di rigore a $x = 1600$ mm, e il bersaglio del rinvio posizionato stabilmente verso centrocampo a $x = 800$ mm.
2. *Risoluzione del bug dei 100 millisecondi:* a causa delle dimensioni ridotte del campo, la palla calciata dall'attaccante impiegava appena 100 ms per varcare la linea



Figura 4: Duello tra attaccante e portiere su `UnicalMini`: posizionamento iniziale con guardia tra i pali (a sinistra) e parata in tuffo laterale (a destra).

dell'area di rigore, mentre il sensore della gamba richiedeva circa 700 ms per registrare il completamento del calcio. L'attaccante interpretava quindi la palla ancora in corsa come una sfera ferma rimasta incastrata in area, attivando prematuramente lo stato di sblocco (`strikerClear`) e correndo addosso al portiere. Il problema è stato risolto dando priorità al vettore velocità del pallone ($v_x > 250$ mm/s), costringendo l'attaccante ad arrestarsi e ad attendere a centrocampo l'esito dell'azione (`strikerWait`).

3. *Sblocco visivo post-gol e riallineamento a* 180°*:* quando il portiere subiva gol, il comando nativo `LookAtBall()` faceva ruotare il collo all'indietro di 90° per continuare a inquadrare la palla in fondo alla rete, impedendogli di vedere il campo per il prosieguo del gioco. L'azione è stata corretta forzando lo sguardo in avanti (`LookForward()`) con scansione attiva (`LookActive`), associata a un riallineamento motorio continuo sul posto a 180° per mantenerlo costantemente rivolto verso il campo.
4. *Sincronizzazione con i tempi di alzata:* a seguito di un tuffo laterale, la routine automatica di rialzata da terra richiede fisiologicamente tra i 6 e gli 8 secondi per preservare i servomotori da sforzi eccessivi. È stato quindi necessario estendere il timer di attesa dell'attaccante fino a 60 secondi, garantendo al portiere il tempo materiale di rialzarsi, raggiungere la palla e completare il rinvio a centrocampo senza subire la pressione anticipata del compagno.

L'architettura complessiva dei due automi cooperativi e le relative transizioni tra gli stati sono schematizzate in Figura 5.

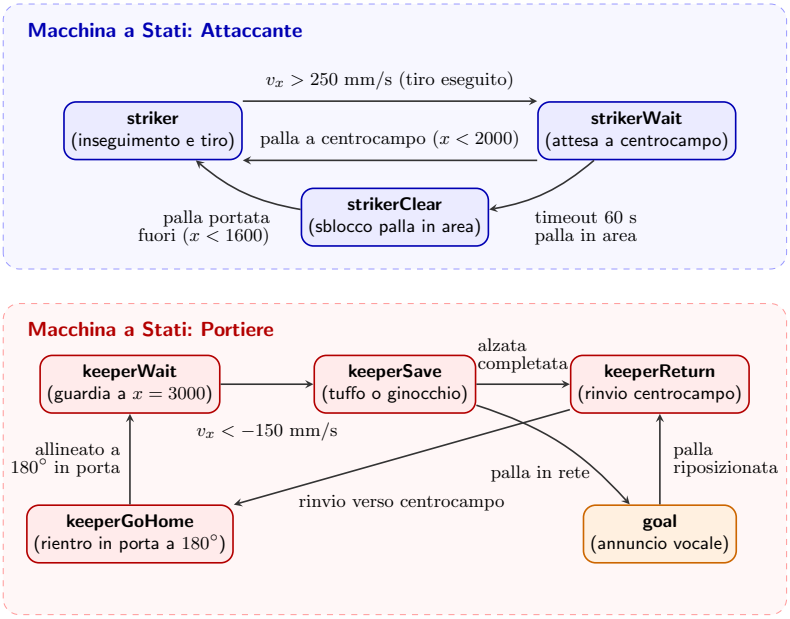


Figura 5: Architettura a stati cooperativa per il duello `DemoStrikerVSKeeperLoopMini`.

3.3 Lo Scambio di Passaggi Continuo a Zone: DemoPassMini

L'obiettivo più ambizioso e originale del lavoro ha riguardato la progettazione da zero di una skill di passaggio continuo cooperativo tra due robot, denominata `DemoPassMini`. A differenza degli scenari con la porta, il passaggio tra due agenti richiede un perfetto sincronismo temporale e spaziale per evitare che i robot entrino in collisione o che l'azione si interrompa dopo pochi scambi.

Nelle prime prove libere è emerso un evidente problema di deriva: ogni volta che un NAO avanza per calciare compie un passo di circa $30-40$ cm; senza un ancoraggio fisso, i due robot finivano per avvicinarsi progressivamente a ogni tocco, fino a calpestarsi dopo tre o quattro passaggi. Per garantire la stabilità del ciclo all'infinito, il mezzo campo è stato suddiviso in due zone di competenza separate dalla linea dell'area di rigore, assegnando a ciascun robot una stazione fissa (*Home Position*):

- *Giocatore 1 (Area Porta):* competente per $x \geq 1600$ mm, con Home posizionata a $x = 2600$ mm orientata a $180°$ verso centrocampo;
- *Giocatore 2 (Area Mezzaluna):* competente per $x < 1600$ mm, con Home posizionata a $x = 400$ mm orientata a $0°$ verso la porta.

La suddivisione geometrica degli spazi e le relative *Home Positions* sono illustrate nello schema spaziale di Figura 6.

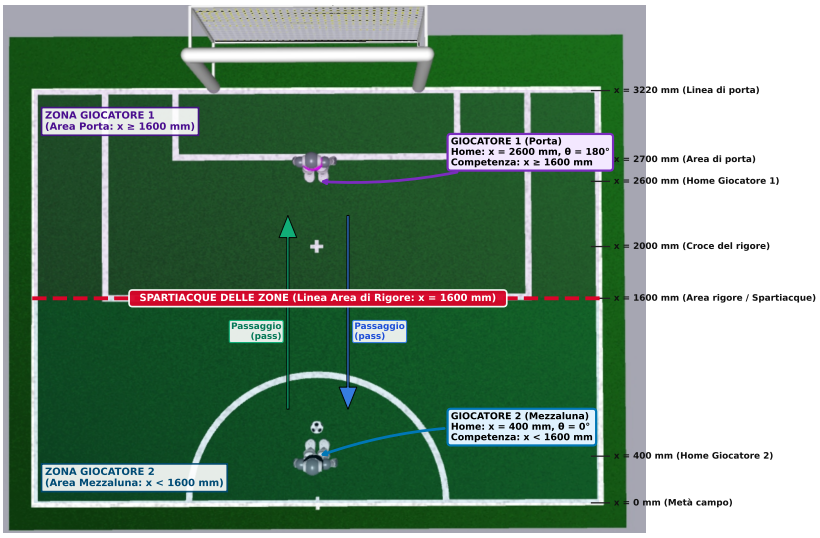


Figura 6: Zonizzazione spaziale e architettura delle *Home Positions* in `DemoPassMini`: spartiacque a $x = 1600$ mm (area di rigore), stazioni fisse dei due NAO e traiettorie dello scambio continuo sul mezzo campo `UnicalMini`.

Terminato il calcio, il robot non resta sul posto ma indietreggia obbligatoriamente verso la propria Home (`returnHome`) mantenendo lo sguardo attivo sulla sfera. Questa mutua esclusione assicura che solo il robot la cui area contiene la palla si muova per giocarla, mentre l'altro attende stazionario.

Risoluzione del "Ball Dodging" e Ricezione a Piedi Saldi

Durante i test sui robot fisici si è manifestata un'anomalia critica: quando il Giocatore 2 effettuava il passaggio, il ricevitore compiva puntualmente dei passi laterali scartando la palla e lasciandola sfilare verso il fondo campo. L'indagine nel codice sorgente di B-Human ha rivelato che la richiesta motoria `shouldWalkOutOfBallLine` era nativamente impostata a `true` ogni volta che la palla rotolava verso la porta avversaria (`isRollingTowardsOpponentGoal`), per evitare che un robot ostacoli involontariamente un tiro in porta del compagno. Poiché su `UnicalMini` la porta avversaria si trova esattamente dietro al Giocatore 1, ogni passaggio in avanti veniva interpretato come un tiro in porta, inducendo il robot a scansarsi. Disabilitando forzatamente questo flag e imponendo la postura a piedi piantati a terra all'avvicinarsi rapido del pallone, il corpo del robot è tornato a fungere da barriera solida che stoppa e controlla la sfera.

Controllo della Potenza del Calcio e Calibrazione Empirica

Uno degli obiettivi principali della skill è garantire che i passaggi siano il più precisi possibile. Un fattore determinante per la precisione è proprio la potenza impressa al tiro: operando su un campo ridotto, un calcio standard risulterebbe eccessivo, facendo

rimbalzare violentemente la palla contro il robot ricevitore per poi allontanarsi da esso e vanificare lo scambio.

Per questa ragione si è scelto di modulare la potenza calcolando la gittata in modo dinamico rispetto alla distanza effettiva tra la palla e il compagno:

$$L_{\text{kick}} = \text{clamp}\left(\|\mathbf{p}_{\text{partner}} - \mathbf{p}_{\text{ball}}\| + 300 \text{ mm},\ 500 \text{ mm},\ 2500 \text{ mm}\right) \tag{1}$$

L'offset di +300 mm serve a compensare l'attrito della moquette del laboratorio, assicurando che la palla arrivi a destinazione senza fermarsi prima.

I valori numerici e le costanti impiegate sono stati scelti in modo empirico durante i test: garantiscono il funzionamento dello scambio, ma non sono i più precisi in assoluto e potranno essere ulteriormente affinati. Inoltre, è emersa una discrepanza tra il simulatore e la realtà per quanto riguarda la forza impressa ai tiri e la risposta del pallone. Di conseguenza, un utile obiettivo futuro sarà effettuare calibrazioni specifiche anche sui parametri fisici di SimRobot, così da rendere il comportamento del simulatore il più fedele possibile alle dinamiche osservate sul campo reale.

Sincronizzazione Temporale: Lo Stato di Attesa Pre-Calcio

Durante i test è emerso che, dopo aver calciato, il robot impiega diversi secondi per rientrare alla propria *Home position* e riallinearsi. Di conseguenza, se il compagno riceveva la sfera e calciava immediatamente, il partner non era ancora pronto a ricevere il passaggio. Per sincronizzare le due azioni è stato introdotto lo stato intermedio `waitToKick`: una volta posizionatosi dietro la sfera, il robot non calcia subito, ma si arresta in piedi per un intervallo di *3 secondi*, continuando a seguire il pallone con la testa. Questo intervallo concede al compagno il tempo necessario per completare il rientro e voltarsi verso il campo; solo allo scadere del timer il robot esegue il passaggio, garantendo la continuità dello scambio.

Comunicazione Wi-Fi e Ricerca del Pallone

Un aspetto didattico centrale di questa skill cooperativa era sfruttare la comunicazione Wi-Fi tra i due robot per condividere le informazioni di gioco. Durante le prove si presentava un problema intuitivo: quando la palla arrivava nell'altra metà campo, il compagno vi si posizionava a ridosso per preparare il tiro. A causa di questa vicinanza, il corpo e le gambe del partner finivano per coprire fisicamente la palla, occludendola alla vista del robot in attesa alla base. Quest'ultimo, non vedendo più la sfera, la riteneva persa e iniziava a girarsi su se stesso per cercarla, finendo per voltare le spalle proprio nel momento in cui il passaggio stava per partire.

Grazie allo scambio dati in rete (`TeammatesBallModel`), il problema è stato risolto: se il compagno comunica via Wi-Fi di vedere la palla nella propria zona, il robot ricevitore comprende che la sfera è solo temporaneamente nascosta dal corpo del partner e resta fermo nella propria posizione, continuando a guardare in avanti. Se invece nessuno dei due robot rileva la palla per oltre 7.5 secondi (ad esempio a causa di una deviazione imprevista), si attiva una ricerca con rotazione sul posto a 360° per riagganciarla.

L'automa a stati finiti complessivo e il ciclo continuo di transizioni della skill sono illustrati nel diagramma di Figura 7.

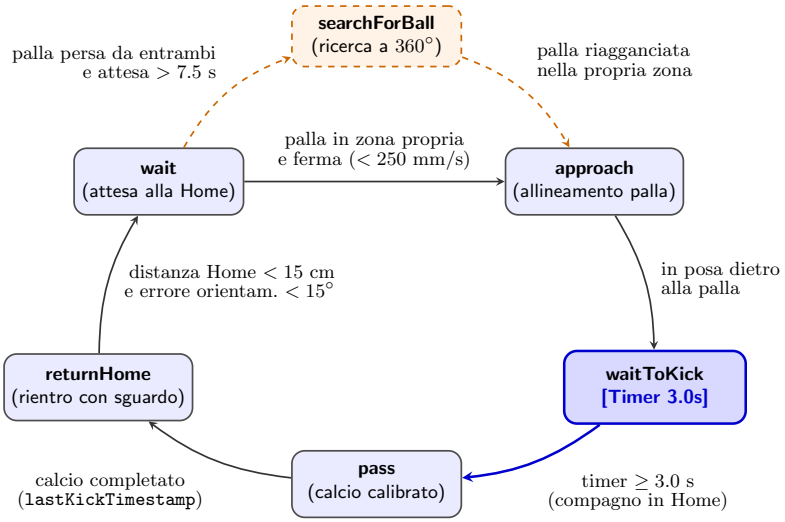


Figura 7: Diagramma della macchina a stati finiti (CABSL) per la skill `DemoPassMini`.

4 Note Operative di Compilazione e Deployment sui Robot

Per trasferire il codice sviluppato a bordo dei NAO fisici, la pipeline di B-Human prevede la compilazione del target `Develop` tramite CMake e Ninja:

```
ninja -C Build/Linux/CMake/Develop
```

Una volta generati i binari, l'invio contemporaneo del software su entrambi i robot avviene mediante lo script di deploy, specificando per ciascun agente il proprio identificativo numerico e l'indirizzo IP all'interno della rete locale del laboratorio:

```
./Make/Common/deploy Develop -s <NomeSkill> -l UnicalMini -v 30 -d -b -r 1 <IP_Robot1> -r 2 <
    IP_Robot2>
```

I parametri impiegati nel comando rispondono alle seguenti esigenze operative:

- `-s <NomeSkill>`: imposta la skill da eseguire all'avvio (ad esempio `StrikerVsKeeperLoopMini` o `DemoPassMini`);
- `-l UnicalMini`: carica a bordo macchina i file di calibrazione geometrica e di localizzazione specifici per il mezzo campo del DIMES;

- `-v 30`: imposta il volume degli altoparlanti al 30%, evitando notifiche vocali a volume eccessivo durante i test prolungati;
- `-d`: elimina i log e i dati temporanei accumulati nella memoria flash eMMC del NAO, risolvendo alla radice l'errore di saturazione disco (`No space left on device`);
- `-b`: riavvia automaticamente il servizio software di B-Human sul robot non appena terminato il trasferimento dei file.

5 Conclusioni e Sviluppi Futuri

Le attività svolte nel corso delle 50 ore dedicate al progetto RoboCup del DIMES hanno consentito di completare con successo l'intero ciclo di lavoro previsto: dal ripristino sistemistico della flotta di robot NAO V6 fino alla progettazione e calibrazione sul campo di comportamenti cooperativi autonomi.

L'introduzione della configurazione dedicata `UnicalMini` ha permesso di superare i vincoli imposti dagli spazi ridotti del laboratorio, formalizzando metricamente il mezzo campo sia a bordo macchina sia nell'ambiente di simulazione SimRobot. Su questa base geometrica è stato possibile prima validare le skill cinematiche a singolo robot (`DemoWalkSquare` e `DemoScoreGoalAndStop`) e successivamente sviluppare i comportamenti continui a due robot: il ciclo di tiro e parata (`DemoStrikerVSKeeperLoopMini`) e lo scambio di passaggi a zone con sincronizzazione temporale e cooperazione Wi-Fi (`DemoPassMini`).

Tra i principali spunti per gli sviluppi futuri emergono:

- Una nuova campagna di rilievo delle dimensioni del mezzo campo condotta con strumenti di misurazione più precisi (come distanziometri laser o bindelle metriche professionali), così da superare le tolleranze fisiologiche dell'applicazione AR per smartphone e affinare ulteriormente i parametri geometrici in `fieldDimensions.cfg`;
- La ricalibrazione dei parametri fisici di SimRobot (attrito volvente e contatto del motore ODE), al fine di ridurre la discrepanza rilevata rispetto alla dinamica reale del pallone sulla moquette del laboratorio;
- L'affinamento delle costanti di potenza e dei tempi di attesa nei calci per ottimizzare ulteriormente la precisione dei passaggi;
- L'estensione dei comportamenti cooperativi a scenari di gioco più articolati, introducendo il posizionamento dinamico in campo e l'intercettazione attiva del pallone in movimento.

Riferimenti bibliografici

[1] Röfer, T., Laue, T., Hasselbring, A., et al.: B-Human Team Report and Code Release 2024. Technical report, Universität Bremen and Deutsches Forschungszentrum für Künstliche Intelligenz (DFKI) (2024). Repository GitHub: https://github.com/bhuman/BHumanCodeRelease/tree/coderelease2024. Documentazione ufficiale: https://docs.b-human.de/coderelease2024/getting-started/running-the-code/